Chapter 2.

Natural Language Processing

1.7. More Embeddings

Syntactic models

Syntactic models focus on the structure and form of language, analyzing how words and symbols are arranged according to grammatical or formal rules.

Semantic models

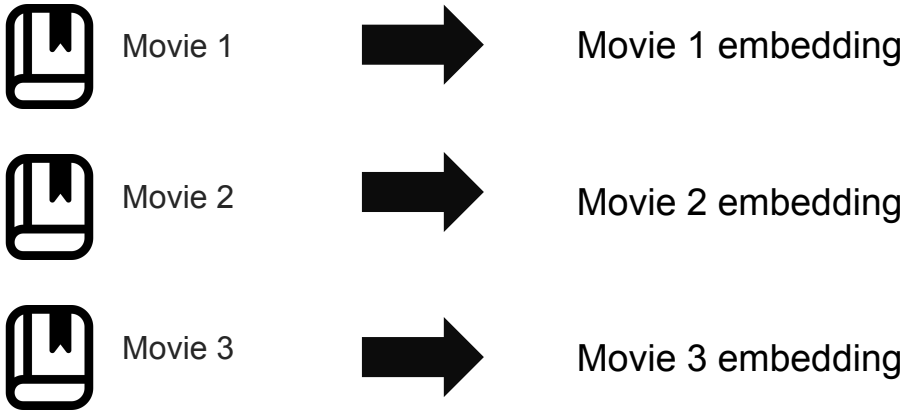
Semantic models capture meaning and relationships between concepts, interpreting the content and context of language beyond its surface structure.

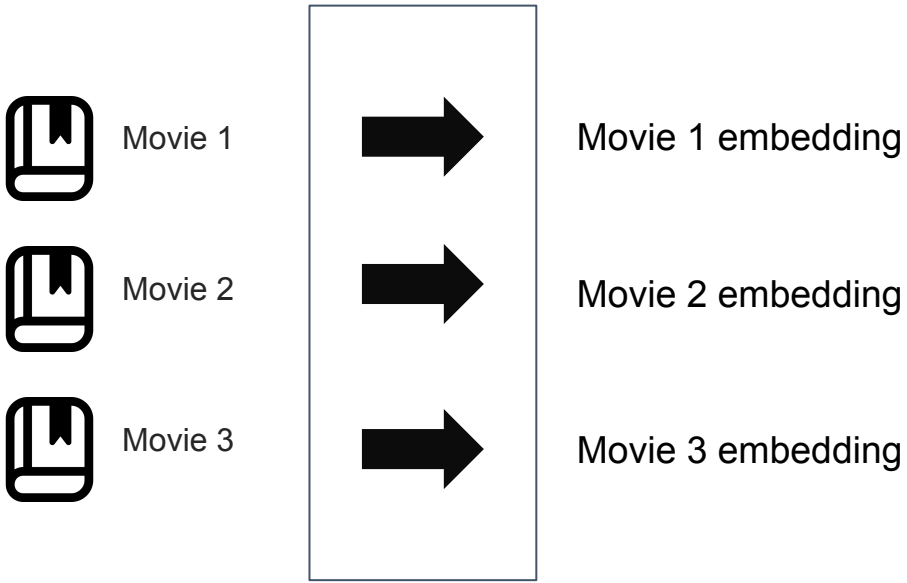
Syntactic models

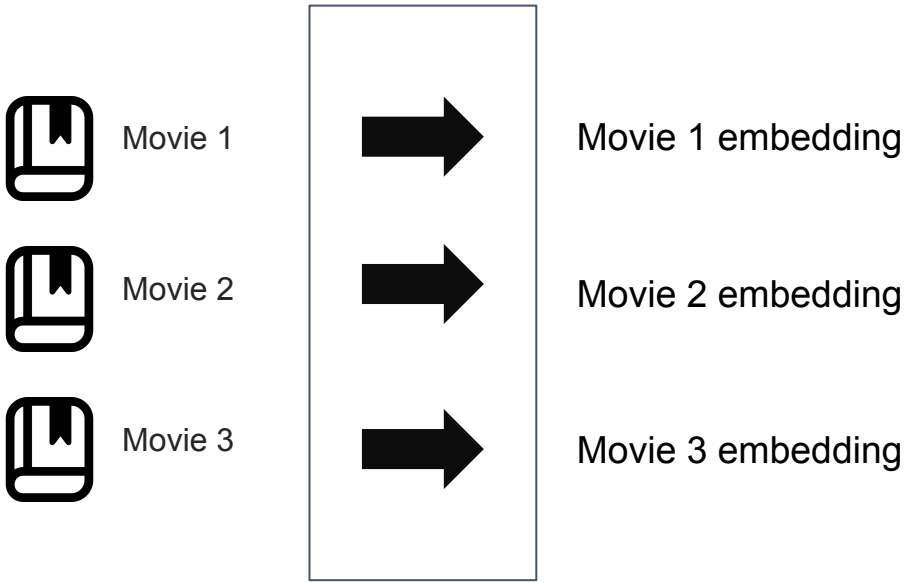
Syntactic models focus on the structure and form of language, analyzing how words and symbols are arranged according to grammatical or formal rules.

Semantic models

Semantic models capture meaning and relationships between concepts, interpreting the content and context of language beyond its surface structure.


Movie 1
Movie 1 embedding
Movie 2
Movie 2 embedding
Movie 3
Movie 3 embedding


Movie 1
Movie 2
Movie 3
Movie 1 embedding
Movie 2 embedding
Movie 3 embedding


Movie 1
Movie 2
Movie 3
Movie 1 embedding
Movie 2 embedding
Movie 3 embedding

Semantic models

Semantic models capture meaning and relationships **between concepts**, interpreting the content and **context** of language beyond its surface structure.

Semantic models

Semantic models capture meaning and relationships **between concepts**, interpreting the content and **context** of language beyond its surface structure.

Words that appear in similar **contexts** have similar meanings

Analogy: Like learning a language - you learn words by seeing them in context

Words that appear in similar **contexts** have similar meanings

Analogy: Like learning a language - you learn words by seeing them in context

CBOW (Continuous Bag of Words)

Skip-gram

INPUT -> Neural Network -> OUTPUT

CBOW (Continuous Bag of Words)

Concept: Context → Predict center word

Analogy: "Fill in the blank"

```
"The quick brown _____ jumps over the lazy dog"

        ↓

    Predict: "fox"


Input: [the, quick, brown, jumps]

Output: fox
```

Skip-gram

Concept: Center word → Predict context

Analogy: "Predict neighbors"

```
Center: "fox"
         ↓
Predict context: [the, quick, brown, jumps]

Input: fox
Output: [the, quick, brown, jumps]
```

Example

```
Text: "I love machine learning"

Window 1: [I, love, machine] → predict "learning"
Window 2: [love, machine, learning] → predict "I"
Window 3: [machine, learning, ...] → predict "love"
...

After millions of windows:
- "machine" and "learning" appear together often
- They get similar vectors!
- "love" appears with positive words
  - It gets a positive sentiment vector!
```

Early algorithms

- Word2Vec
- Glove

Early algorithms

Word2Vec

1. Start with random vectors for all words
   (Like random guesses - they'll get better!)

2. Slide a window through text:
   "The quick brown fox jumps over the lazy dog"
    [---- window ----]
    (Look at a few words at a time)

3. For each window:
   - CBOW: Given "the quick brown jumps", predict "fox"
   - Skip-gram: Given "fox", predict "the quick brown jumps"

4. If prediction is wrong, adjust the vectors
   (Move them closer together or farther apart)

5. Repeat millions of times
   (Go through entire Wikipedia, news articles, etc.)

6. Result: Words that appear together get similar vectors!

Early algorithms

GloVe

1. Build co-occurrence matrix:
   Count how often words appear together across entire corpus

   Example:

	the	cat	sat	mat
the	0	100	50	30
cat	100	0	80	10
sat	50	80	0	70
mat	30	10	70	0

2. Factorize matrix into word vectors
3. Words that co-occur frequently → similar vectors

We are learning word embeddings.
Not sentences.
Not documents.

```
We are learning word embeddings.
Not sentences.
Not documents.
```

?

What if we have sentences or documents?

We are learning word embeddings.
Not sentences.
Not documents.

?

What if we have sentences or documents?
What are the features?

We are learning word embeddings.
Not sentences.
Not documents.

?

Each word has a vector.
A sentence or a document is a collection of words.

We are learning word embeddings.
Not sentences.
Not documents.

?

Average the words?
Give relative importance?

We are learning word embeddings.
Not sentences.
Not documents.

?

How relevant is pre-processing?

We are learning word embeddings.
Not sentences.
Not documents.

?

How relevant is pre-processing?
(one vector per word, so still very relevant)

Only one learning notebook this time :D

But we can learn sentence/document embeddings.

But we can learn sentence/document embeddings.

Most if not all modern language models learn at the sentence level.

The most ‘known’ family of models are BERT and now the GPT-based ones.

But we can learn sentence/document embeddings… by

- **Predicting what's in between (similar to CBOW)**
- **Predicting what's next**

Both have to first learn the context to understand what might be missing.

- **Predicting what's similar**

But we can learn sentence/document embeddings… by

- **Predicting what's in between (Masked Language Modeling)**
- **Predicting what's next (next-token prediction, GPT style)**

Both have to first learn the context to understand what might be missing.

- **Predicting what's similar (contrastive learning)**

LEARN = Adjust the vectors i.e adjust the representation of a given document/sentence/concept based on a given objective

- **Predicting what’s in between (Masked Language Modeling)**

predict missing words from context

- **Predicting what's in between (Masked Language Modeling)**

predict missing words from context

Just like CBOW?

Key Differences:

Example: Sentence: "The cat sat on the mat"

- **CBOW**: Uses just ["cat", "on"] → predict "sat"
- **MLM**: Uses ["The", "cat", "[MASK]", "on", "the", "mat"] → predict "sat"

Bottom line: MLM is like "CBOW 2.0" - same core idea, but sees the whole sentence and understands context better!

- **Predicting what’s next (next-token prediction, GPT style)**

- **Predicting what's next (next-token prediction, GPT style)**

The Idea: Guess what comes next!

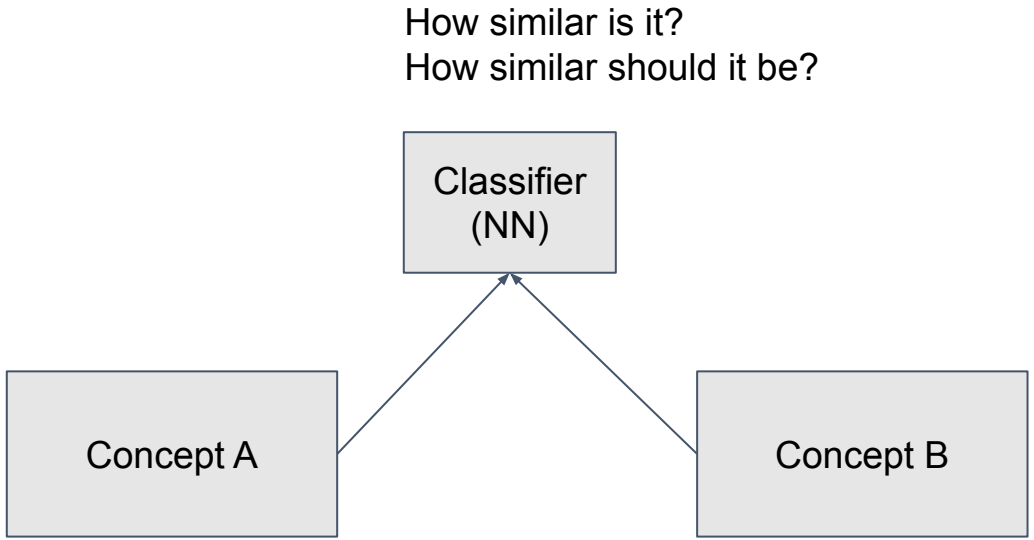
Example: "The cat sat on the" → predict **"mat"**

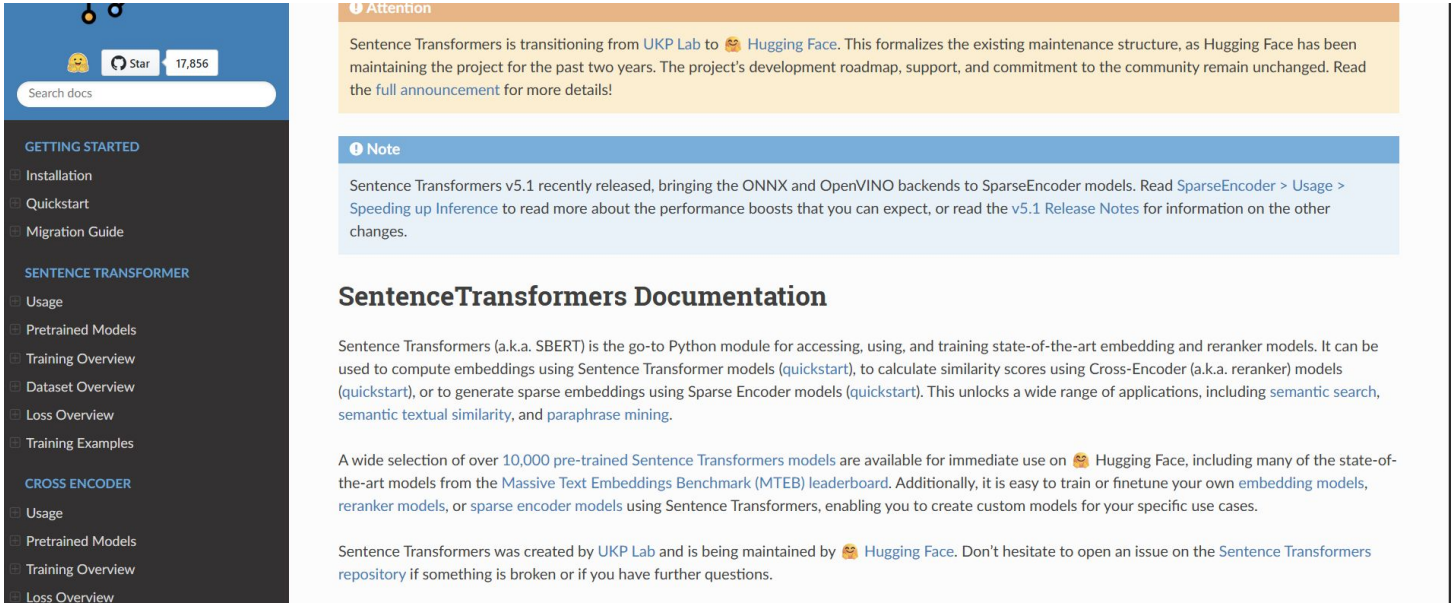
The model learns embeddings as a side effect

Example: Input: "The cat sat on the mat" ↓ Model processes → internal vectors for understanding
↓ Extract these vectors = **embeddings!**

- **Predicting what’s similar (contrastive learning)**

- **Predicting what's similar (contrastive learning)**





https://www.sbert.net/

The Big Idea:
Computers only work with numbers. We gotta convert text to numbers.
How we convert it matters a lot!

Key Points:

1. **TF-IDF (Old Way): Sparse vectors, mostly zeros. "great" ≠ "excellent"**
   Tool: scikit-learn

2. **Word Embeddings (Better): Dense vectors. "great" ≈ "excellent"**
   Tools: gensim (Word2Vec, GloVe, FastText)

3. **Sentence Embeddings (Best): Captures word order and context**
   Tool: sentence-transformers (Sentence-BERT)

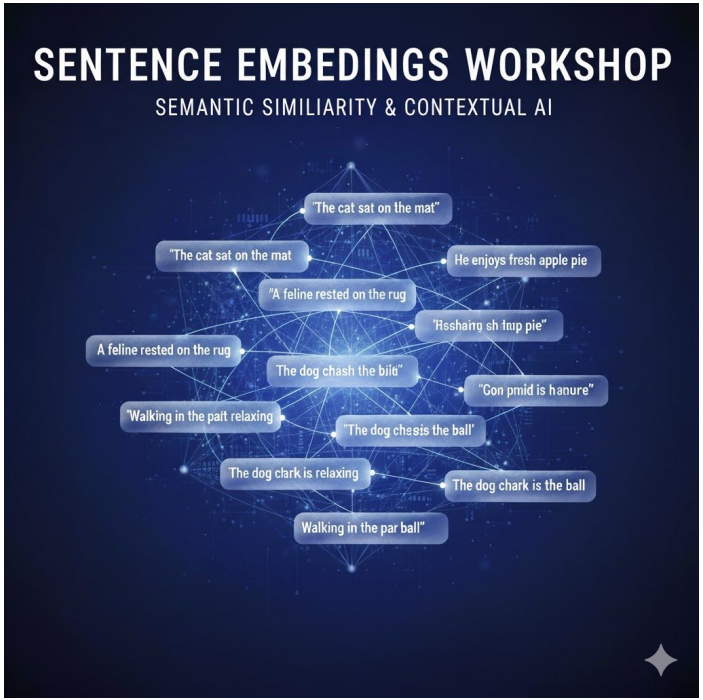
4. **LEARN = Adjust the Vectors: Three ways to learn**
   - GPT: Predict next word
   - BERT: Predict missing word
   - Sentence-BERT: Predict similarity

5. Preprocessing: Clean text first
   Tools: spaCy, NLTK

6. Visualization: t-SNE/PCA/UMAP shows similar words cluster together
   Tools: matplotlib, seaborn

7. Applications:
   TF-IDF → Keyword Search
   Word Embeddings → Semantic Search
   Sentence Embeddings → Classification

The Journey:
TF-IDF (word patterns) → Word Embeddings (word meaning) → Sentence Embeddings (sentence meaning)


SENTENCE EMBEDINGS WORKSHOP
SEMANTIC SIMILIARITY & CONTEXTUAL AI
"The cat sat on the mat"
"The cat sat on the mat
He enjoys fresh apple pie
"A feline rested on the rug
"Hsshairg sh tnup pie"
A feline rested on the rug
The dog chash the bile"
"Con pmid is hanure"
"Walking in the pait relaxing
"The dog chesis the ball'
The dog clark is relaxing
The dog chark is the ball
Walking in the par ball"